"Advanced Statistics for Physics Analysis"
Laboratory session 3 - report

Libero Pollini

June 5, 2026

Overview

We analyze a binary-response dataset using Bayesian Bernoulli regression. The data consist of observations

$$D = \{(y_i, x_i)\}_{i=1}^{N} \tag{1}$$

where $y_i \in \{0, 1\}$ is a binary outcome and x_i is a real-valued explanatory variable. We fit Bayesian logistic regression models using the R library `Stan`[1] and explore its diagnostics. A first visualization of the data is provided in Figure 1, where we get an impression of an increase in probability of success $P(y = 1|x)$ as x increases. We also note that this is a small dataset ($N = 120$).

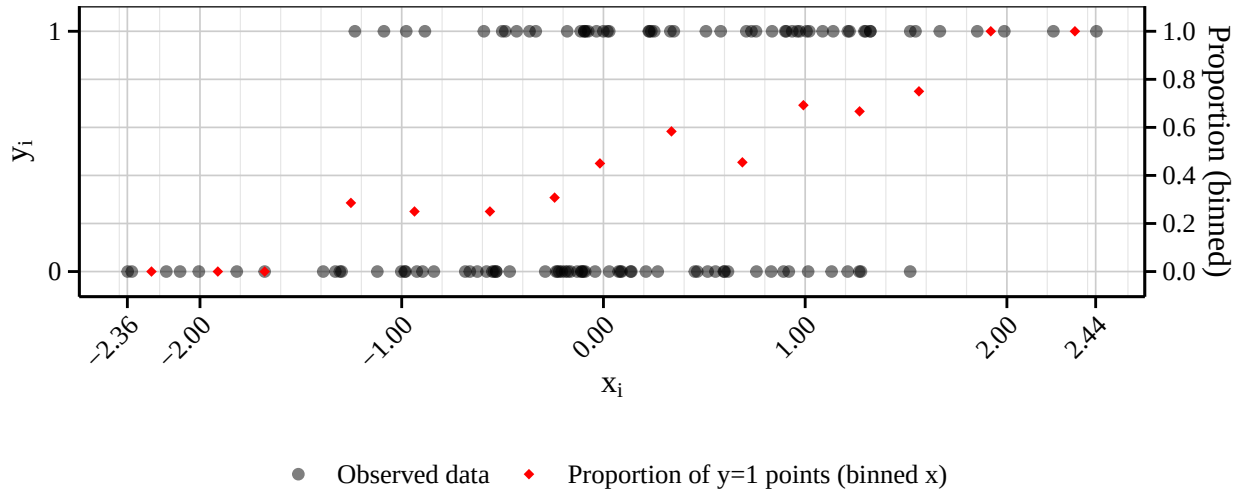


Figure 1: *Binary outcome y_i vs. explanatory variable x_i. x_i is binned into 15 bins; each red dot's coordinates are the mean x for that bin, and the number of points with $y_i = 1$ in the bin.*

Model

The main model, which we name M_1, is defined as

$$y_i|\beta_0, \beta_1 \sim \text{Bernoulli}(\pi_i), \tag{2}$$

$$\text{logit}(\pi_i) = \beta_0 + \beta_1 x_i, \tag{3}$$

[1] Stan Development Team. *Stan Modeling Language Users Guide and Reference Manual*. `https://mc-stan.org`, 2026.

with independent (and weakly informative) priors

$$\beta_0 \sim N(0, 3^2), \tag{4}$$

$$\beta_1 \sim N(0, 3^2). \tag{5}$$

Equivalently,

$$\pi_i = \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)}. \tag{6}$$

1 Posterior formulation and implementation

Below is the `.stan` file code which we used to build the main model:

```
data {
int<lower=1> N;
vector[N] x;
array[N] int<lower=0, upper=1> y;
}

parameters {
real b0;
real b1;
}

model {
b0 ~ normal(0, 3);
b1 ~ normal(0, 3);
y ~ bernoulli_logit(b0 + b1 * x);
}

generated quantities {
array[N] int y_rep;
for (i in 1:N)
y_rep[i] = bernoulli_logit_rng(b0 + b1 * x[i]);

vector[N] log_lik;
for (i in 1:N)
log_lik[i] = bernoulli_logit_lpmf(y[i] | b0 + b1 * x[i]);
}
```

2 MCMC fitting and diagnostic

First, we standardize the x variable beforehand, to reduce posterior correlations (due to geometry when x has a mean different from 0) between parameters and to make the prior scale invariant (by making the SD of the new x equal to 1). Using the above model, we then run 4 parallel Monte Carlo Markov Chains (MCMC), each of 2000 iterations ("length"), preceded by a burn-in period of 1000 iterations. (Execution takes only a few seconds on a single-core CPU.) By default, `Stan` uses a Hamiltonian Monte Carlo (HMC), No-U-Turns (NUTS) algorithm, automatically adapting step size and the mass matrix during warmup, and drawing initial parameters (if unconstrained, as in our case) from a uniform distribution between -2 and 2. `Stan`'s diagnostics (Table 1) show that:

- the chains have converged, as Vehtari-Gelman's $\hat{R}$ statistic[2] is effectively 1;
- they mixed well, as the effective sample size (ESS) both in the bulk and in the tails of the distribution is comparable to the total number of posterior draws, 8000;

[2]Vehtari et al. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2)

- the uncertainty on the parameters (standard deviation, SD, and mean absolute deviation, MAD) is also acceptable. In particular, the final SD is much smaller than that chosen for the original prior on each parameter (3.0).

Furthermore, the output of `cmdstan_diagnose()` states that:

- no divergent transitions (*i.e*, iterations where the numerical error on the Hamiltonian is extremely large) were found;
- satisfactory "tree depth" was reported, *i.e.* the maximum depth (default is 10) of tree for the NUTS algorithm was never reached, indicating that the sampler had sufficient trajectory length to propose distant states;
- satisfactory Energy Bayesian Fraction of Missing Information (E-BFMI) was reported, *i.e.* the variance (over chain samples) of the energy is close to that expected if the posterior was well-conditioned, indicating the absence of narrow "canyons" or "funnels".

Note that $\hat{R}$ and ESS are calculated on the "rank normalized", "split" chains. This means that 1) all samples are mapped to their empirical cumulative distribution function (CDF), then mapped back to a standard normal distribution via the standard normal inverse CDF function; then, 2) each half chain is considered separately. Finally, the autocorrelation plot (Figure 4) shows that the autocorrelation function (ACF) decays quickly for both parameters, indicating again that the chains have mixed well. Trace plots (Figure 2) also confirm what we have stated so far.

Table 1: *Posterior summaries of main model's parameters.*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
β_0	-0.3	-0.3	0.2	0.2	-0.6	0.1	1.00	6203	5162
β_1	1.0	1.0	0.2	0.2	0.6	1.4	1.00	6367	5282

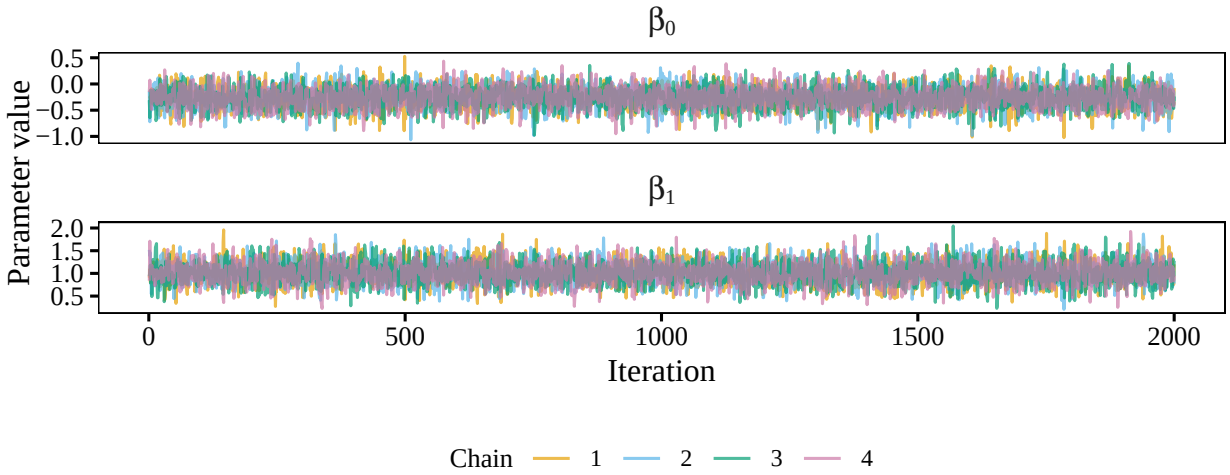


Figure 2: *Trace plots for β_0 and β_1 (full model M_1).*

3 Posterior inference

Full posterior summaries have been reported in Table 1. We obtain, with a 95% credible interval (CI):

$$\beta_0 \in [-0.65, 0.15], \quad \beta_1 \in [0.54, 1.50] \tag{7}$$

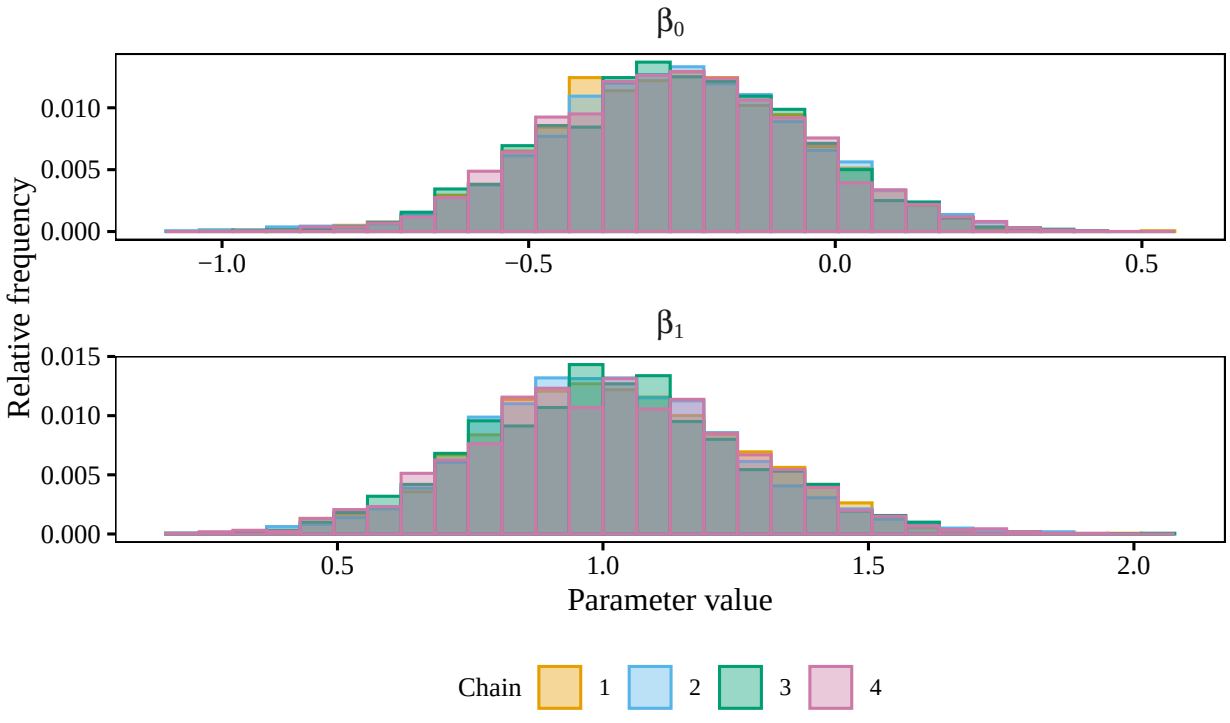


Figure 3: *Marginal posterior distributions (full model M_1).*

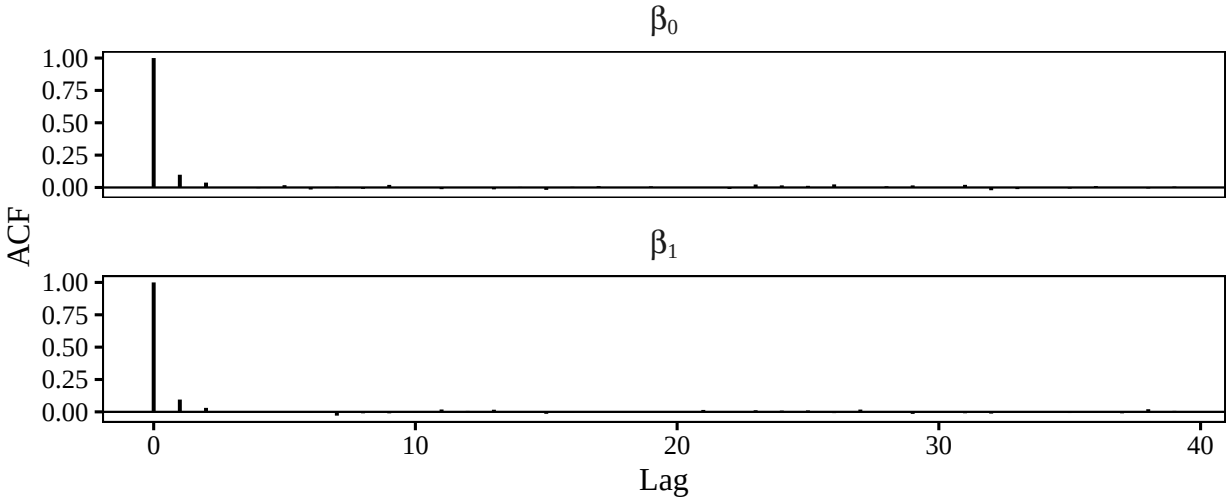


Figure 4: *Autocorrelation functions (full model M_1).*

We observe that the covariate effect β_1 is, with high credibility, non-zero and positive, as expected from our initial data exploration. This can also be observed in Figure 6, where we plot the posterior probability of success with a 95% CI (uncertainty the posterior itself, due to parameter uncertainty, not a *predictive* interval for a new observation), calculated by means of the 2.5% and 97.5% quantiles of the posterior samples. Marginal posterior distributions are reported in Figure 3, where we can appreciate their fairly symmetrical shape. Joint posterior samples are shown in Figure 5, where we observe a negligible correlation (numerically we obtained $\text{COR}(\beta_0, \beta_1) \approx -0.092$) between the two model parameters.

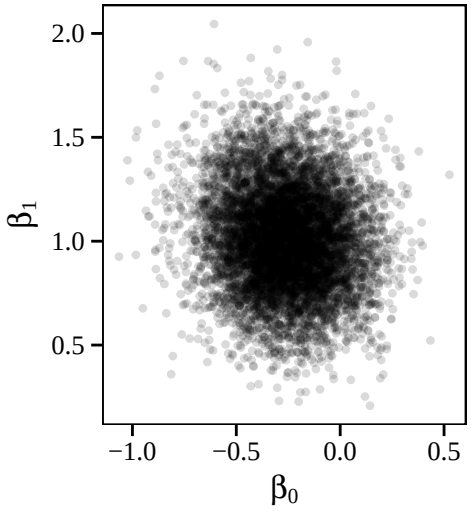


Figure 5: *Joint posterior samples* (β_0, β_1).

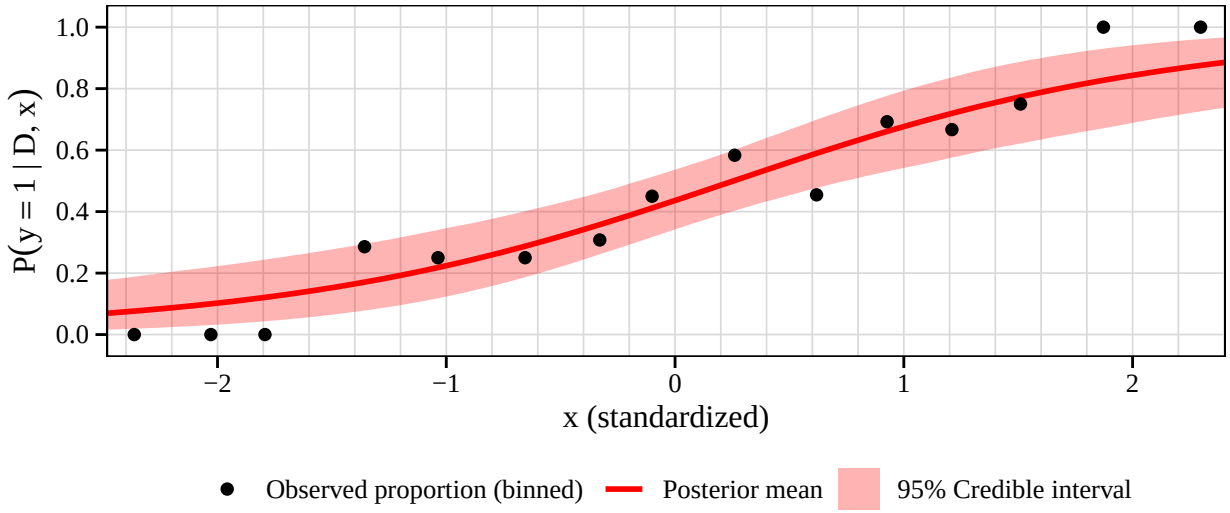


Figure 6: *Posterior predictive probability* $P(y = 1 \mid x, D)$ *with 95% credible interval.*

4 Posterior predictive inference

We now use the posterior simulation (*i.e.*, the MC chain samples) to approximate the posterior predictive distribution for a fixed new value of the predictive variable, $x_{\text{new}} := 0$. Upon defining $\pi_{\text{new}} \equiv P(y_{\text{new}} = 1 \mid$

$x_{\text{new}}, \beta_0, \beta_1$), we obtain with a 95% CI (centered on the mean, as there is enough symmetry to do so) that:

$$\pi_{\text{new}} = (42 \pm 10)\% \tag{8}$$

which is compatible with the empirical mean of successes, $y_{\text{mean}} \approx 45\%$, as expected since $x_{\text{new}} = 0$ is close to x_{mean}. The posterior predictive probability $P(y_{\text{new}} = 1 \mid x_{\text{new}}, D)$ is the same as the mean π_{new} we just obtained, since:

$$P(y_{\text{new}} = 1 \mid x_{\text{new}}, D) = \int_{\text{supp}(\boldsymbol{\beta})} \underbrace{P(y_{\text{new}} = 1 \mid x_{\text{new}}, \boldsymbol{\beta})}_{\pi_{\text{new}}(\boldsymbol{\beta})} \, p(\boldsymbol{\beta} \mid D) \, d\boldsymbol{\beta} = E_{\boldsymbol{\beta}|D}[\pi_{\text{new}}]. \tag{9}$$

For a visualization of this posterior predictive probability *for any value of* x, one can look back at Figure 6 (red line only). Note that in that plot, we omitted to show uncertainty on the predicted outcome: the reason is that, since the outcome is binary, any CI would include either the outcome $y = 1$, the outcome $y = 0$, or both; furthermore, the third was our case when setting reasonably high CIs. Finally, in Figure 7 we plot a histogram of the posterior draws of π_{new} (each of them calculated from a chain sample), overlaid with its default Kernel Density Estimate (KDE).

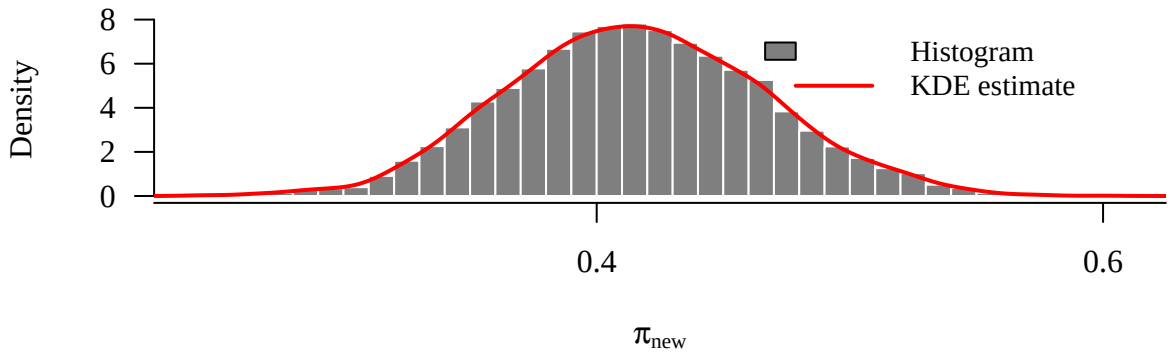


Figure 7: *Posterior draws of* π_{new} *at* $x_{new} = 0$.

5 Model comparison

We now compare the full model M_1 with an intercept-only model M_0:

$$M_0 : \text{logit}(\pi_i) = \beta_0, \quad \beta_0 \sim \mathcal{N}(0, 3^2), \tag{10}$$

$$\tag{11}$$

The `.stan` code is, in this case:

```
data {
int<lower=1> N;
vector[N] x;
array[N] int<lower=0, upper=1> y;
}

parameters {
real b0;
```

```
}

model {
b0 ~ normal(0, 3);
y ~ bernoulli_logit(b0);
}

generated quantities {
array[N] int y_rep;
for (i in 1:N)
y_rep[i] = bernoulli_logit_rng(b0);

vector[N] log_lik;
for (i in 1:N)
log_lik[i] = bernoulli_logit_lpmf(y[i] | b0);
}
```

Following an identical procedure, we obtain good diagnostics (we only note a slightly lower ESS of 3011 and 3961 for bulk and tail respectively; this is probably due to the target distribution being more spread out). Diagnostics and results are shown in Figure 8. Posterior predictive distribution is shown in Figure 9.

We perform model comparison using leave-one-out cross-validation (LOO) via the expected log pointwise predictive density (ELPD)[3]. In a nutshell, the total ELPD in the LOO framework is given by the sum over all y_i of the log leave-one-out predictive probability ELPD_i, *i.e.* the predictive probability of obtaining y_i at coordinate x_i given that the model was fitted on all observations except for the i-th one. In computational practice, as training N models is obviously unfeasible, total ELPD is approximated by means of a weighted sum on predictive probabilities, using importance weights derived from the full-data posterior (from the one fitted model). A further issue is weight instability in the tails of the distribution (large weights): for this reason, tail weights are fitted to a Pareto distribution (this is called Pareto smoothed importance sampling or PSIS), whose estimated parameter $\hat{k}$ is used to determine whether the metric is sound for a given problem. In our case, upon checking diagnostics (in particular, a Pareto $\hat{k} < 0.7$ for both models), we deem the metric suitable. We observe that M_1 has a substantially higher ELPD than M_0, with the difference exceeding twice its standard error, providing strong evidence that the covariate x_i improves predictive performance. Once again, this reflects our most obvious expectations: data is seen to depend positively on the x variable, so an intercept-only model is expected to perform poorly.

6 Conclusion

The four chains converged rapidly and mixed well, as confirmed by $\hat{R} \approx 1$, high ESS, negligible autocorrelation, and clean trace plots. For the main model M_1, posterior uncertainty on both parameters is moderate (SD ≈ 0.2), and the prior is substantially updated. The covariate effect β_1 is credibly positive and non-zero (95% CI excludes zero), confirming that x_i increases the probability of success. The posterior predictive probability at $x_{\text{new}} = 0$ is $42 \pm 10\%$. Model comparison via LOO-ELPD strongly favors M_1 over the intercept-only model M_0 (difference 9.9, more than twice the standard error 4.2), confirming that including the covariate substantially improves predictive performance.

[3] Vehtari, Gelman & Gabry (2017). Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing*, 27:1413–1432.

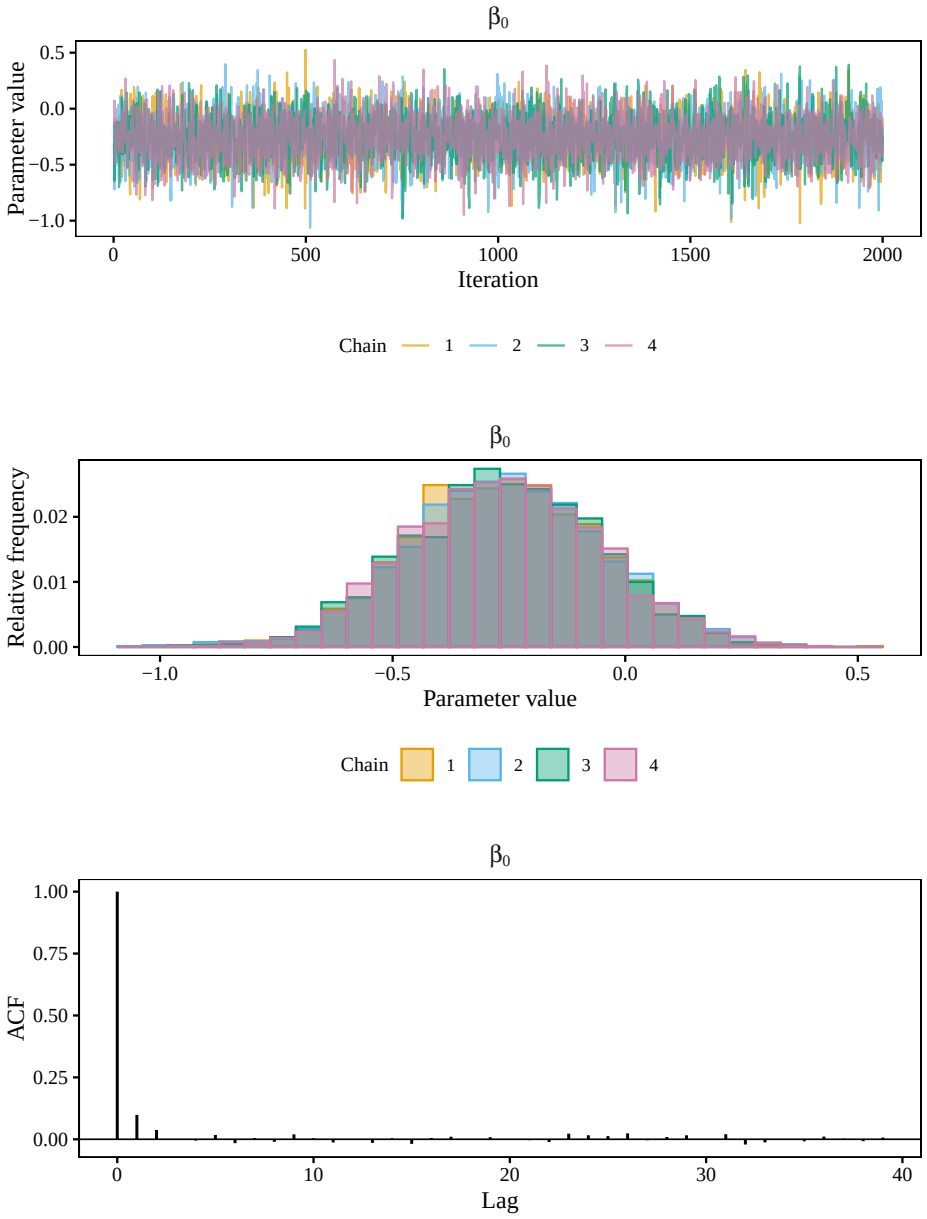


Figure 8: ***Diagnostics for*** M_0. *Top: trace plot. Middle: posterior distribution. Bottom: autocorrelation function.*

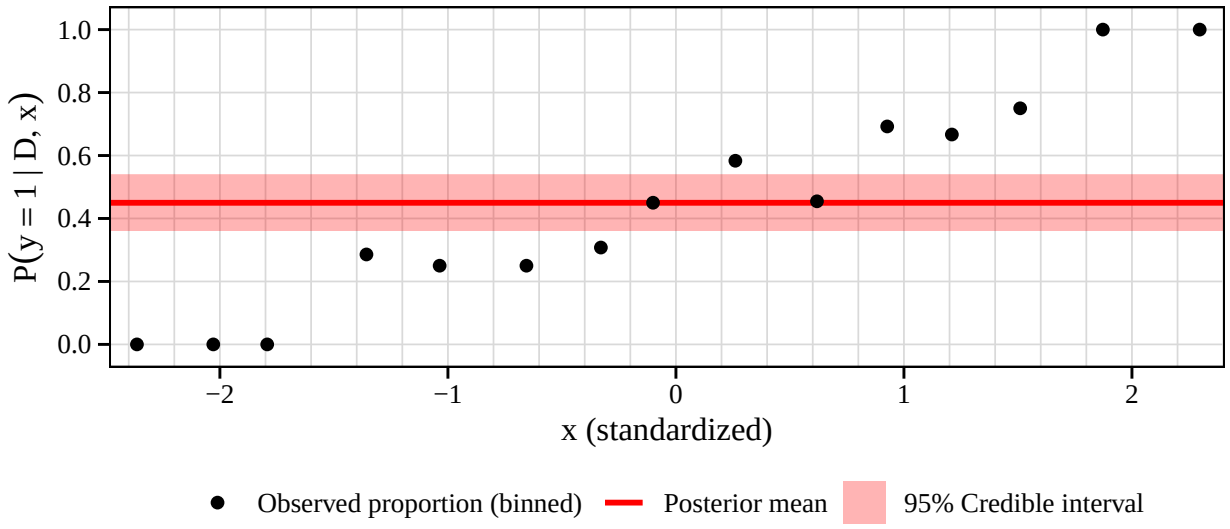


Figure 9: *Posterior predictive probability for intercept-only model M_0.*

R code

The complete R code (for all sections) follows, commented where necessary. Output, where needed, is also included as comments. (`.stan` files code is not included here.)

```
# for correct compilation of stan files (make.exe)
Sys.setenv(PATH = paste("C:/rtools44/usr/bin", Sys.getenv("PATH"), sep = ";"))

library(ggplot2)
library(cmdstanr)
library(posterior)
library(loo) # for model comparison

# import dataset
full_data <- read.csv("C:/Users/Libero/uni/1B/adv_stats/assignment_3/logistic_data.csv", header = TRUE)
x_data <- full_data$x
y_data <- full_data$y
# explore it
typeof(x_data)
# [1] "double"
typeof(y_data)
# [1] "integer"
cat(min(x_data), max(x_data))
#-2.35841 2.443902
cat(min(y_data), max(y_data))
N <- length(x_data)
cat(N)
# 120
x_mean <- mean(x_data)
cat(x_mean)
# 0.08112576
x_std <- sd(x_data)
cat(x_std)
# 0.9821174
y_mean <- mean(y_data)
cat(y_mean)
# 0.45
y_std <- sd(y_data)
cat(y_std)
# 0.4995797

# standardize x variable
```

```r
x_stdd <- (x_data - x_mean) / x_std
stan_data_sdd <- list(N = N, x = x_stdd, y = y_data)

# plot raw data
df_plot <- data.frame(x = x_data, y = y_data) # dataframe

n_bins <- 15
df_plot$bin <- cut(df_plot$x, breaks = n_bins) # bin x in n_bins
binned <- aggregate(y ~ bin, data = df_plot, FUN = mean)
bin_midpoints <- tapply(df_plot$x, df_plot$bin, mean) # x coordinate as mean of x's in each bin
binned$x_mid <- as.numeric(bin_midpoints)
colnames(binned)[2] <- "prop"
# x-axis breaks
major_breaks <- c(round(min(x_data), 2), -2, -1, 0, 1, 2, round(max(x_data), 2))
major_breaks <- sort(unique(major_breaks))
minor_breaks <- seq(floor(min(x_data)), ceiling(max(x_data)), by = 0.2)
# Fine y positions for grid lines
y_grid_pos <- seq(0, 1, by = 0.2)
ggplot() +
  # Manual horizontal grid lines
  geom_hline(yintercept = y_grid_pos, color = "grey80", linewidth = 0.3) +
  # Data points
  geom_point(
    data = df_plot, aes(x = x, y = y, color = "Observed data"),
    size = 1.5, alpha = 0.5
  ) +
  geom_point(
    data = binned, aes(x = x_mid, y = prop, color = "Proportion of y=1 points (binned x)"),
    size = 1.5, shape = 18
  ) +
  scale_y_continuous(
    name = expression(y[i]),
    breaks = c(0, 1),
    limits = c(-0.05, 1.05),
    sec.axis = sec_axis(
      ~.,
      name = "Proportion (binned)",
      breaks = y_grid_pos,
      labels = scales::label_number(accuracy = 0.1)
    )
  ) +
  scale_x_continuous(breaks = major_breaks, minor_breaks = minor_breaks) +
  scale_color_manual(
    name = NULL,
    values = c("Observed data" = "black", "Proportion of y=1 points (binned x)" = "red")
  ) +
  labs(
    x = expression(x[i])
  ) +
  theme_bw(base_size = 12, base_family = "serif") +
  theme(
    # Keep x grid lines, remove y grid lines
    panel.grid.major.y = element_blank(),
    panel.grid.minor.y = element_blank(),
    panel.grid.major.x = element_line(color = "grey80", linewidth = 0.3),
    panel.grid.minor.x = element_line(color = "grey90", linewidth = 0.2),
    axis.line = element_line(color = "black", linewidth = 0.5),
    axis.ticks = element_line(color = "black", linewidth = 0.5),
    axis.ticks.length = unit(0.15, "cm"),
    axis.title = element_text(size = 12, family = "serif"),
    axis.text = element_text(size = 10, color = "black", family = "serif"),
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom",
    legend.title = element_blank(),
    legend.text = element_text(size = 10, family = "serif")
  )

# function to compile and fit both stan models
build_fit_stan_model <- function(stan_file_path, my_data) {
  my_mod <- cmdstan_model(stan_file_path)
```

```r
  cat("MODEL COMPILED SUCCESSFULLY\n")
  my_fit <- my_mod$sample(
    data = my_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 2000,
    seed = 222 # for reproducibility
  )
  return(list(mod = my_mod, fit = my_fit))
}

res <- build_fit_stan_model("C:/Users/Libero/uni/1B/adv_stats/assignment_3/bernoulli_model.stan", stan_data_sdd) # chain takes a
mod <- res$mod
fit <- res$fit

fit$summary(c("b0", "b1"))
# # A tibble: 2 × 10
#   variable   mean median    sd   mad     q5    q95  rhat ess_bulk ess_tail
#   <chr>     <dbl>  <dbl> <dbl> <dbl>  <dbl>  <dbl> <dbl>    <dbl>    <dbl>
# 1 b0       -0.260 -0.260 0.207 0.209 -0.594 0.0807  1.00    6203.    5162.
# 2 b1        1.01   1.01  0.244 0.246  0.625 1.42    1.00    6367.    5282.

fit$cmdstan_diagnose()
# Checking sampler transitions treedepth.
# Treedepth satisfactory for all transitions.

# Checking sampler transitions for divergences.
# No divergent transitions found.

# Checking E-BFMI - sampler transitions HMC potential energy.
# E-BFMI satisfactory.

# Rank-normalized split effective sample size satisfactory for all parameters.

# Rank-normalized split R-hat values satisfactory for all parameters.

# Processing complete, no problems detected.

# function to plot chain samples on par1-par2 plane (main model only)
plot_2_correlations <- function(my_draws_df, params_list) {
  cm <- cor(as.matrix(my_draws_df[, params_list])) # correlation matrix
  p <- ggplot(my_draws_df, aes(x = .data[[params_list[1]]], y = .data[[params_list[2]]])) +
    geom_point(alpha = 0.15, size = 0.8, color = "black") +
    coord_fixed(ratio = 1) + # to keep correct proportions (visual clue)
    labs(
      x = expression(beta[0]),
      y = expression(beta[1])
    ) +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      axis.ticks = element_line(color = "black", linewidth = 0.5),
      axis.ticks.length = unit(0.15, "cm"),
      axis.title = element_text(size = 12),
      axis.text = element_text(size = 10, color = "black")
    )
  return(list(cor = cm, plot = p))
}

draws_df <- as_draws_df(fit$draws()) # draws from posterior: dataframe
lapply(draws_df[, c("b0", "b1")], quantile, probs = c(0.025, 0.975)) # quantiles for 95% CI
# £b0
#       2.5%      97.5%
# -0.6535741  0.1452110

# £b1
#     2.5%    97.5%
# 0.543916 1.504841
```

```r
cor_plot <- plot_2_correlations(draws_df, c("b0", "b1"))
print(cor_plot$plot)
cor_plot$cor # correlation matrix: abs value in 0,1 (corr = covariance / product of variances)
#             b0          b1
# b0  1.00000000 -0.09151987
# b1 -0.09151987  1.00000000

# function to  plot posterior: mean, 95% CI, comparison to data
plot_posterior <- function(my_draws_df, my_x_stdd, my_y_data,
                           my_x_grid = NULL, n_grid = 100, n_bins = 15) {
  if (is.null(my_x_grid)) { # custom grid optional
    x_grid <- seq(min(my_x_stdd), max(my_x_stdd), length.out = n_grid)
  } else {
    x_grid <- my_x_grid
  }

  # matrix: rows correspond to (beta0, beta1) samples, columns to the x grid
  S <- nrow(my_draws_df)
  pi_mat <- matrix(NA, nrow = S, ncol = length(x_grid))

  # uncertainty on predicted outcome - not used
  # y_rep <- rbinom(S, size = 1, prob = pi_mat[, i])

  # to consider only b0 if model is the "intercept-only"
  if (!"b1" %in% colnames(my_draws_df)) {
    b1_vals <- rep(0, S)
  } else {
    b1_vals <- my_draws_df$b1
  }

  # porbability of success value for each matrix element
  # plogis is the inverse logit: plogis(a) = 1/(1+exp(-a))
  for (i in seq_along(x_grid)) {
    pi_mat[, i] <- plogis(my_draws_df$b0 + b1_vals * x_grid[i])
  }

  # ... get its mean and quantiles; apply to columns ("2")
  pi_mean <- apply(pi_mat, 2, mean)
  pi_lo <- apply(pi_mat, 2, quantile, 0.025)
  pi_hi <- apply(pi_mat, 2, quantile, 0.975)

  # Bin observed data as in the beginning
  data_binned <- data.frame(x = my_x_stdd, y = my_y_data)
  data_binned$bin <- cut(data_binned$x, breaks = n_bins)
  binned_props <- aggregate(y ~ bin, data = data_binned, FUN = mean)
  binned_mids <- aggregate(x ~ bin, data = data_binned, FUN = mean)
  binned_df <- data.frame(x_mid = binned_mids$x, prop = binned_props$y)

  plot_df <- data.frame(x_stdd = x_grid, mean = pi_mean, lo = pi_lo, hi = pi_hi)

  # Define x axis limits based on observed data
  x_min_obs <- min(my_x_stdd)
  x_max_obs <- max(my_x_stdd)

  # Fine grid positions (every 0.2) for x and y (but restrict grid to observed range)
  x_grid_pos <- seq(floor(x_min_obs), ceiling(x_max_obs), by = 0.2)
  x_grid_pos <- x_grid_pos[x_grid_pos >= x_min_obs & x_grid_pos <= x_max_obs]
  y_grid_pos <- seq(0, 1, by = 0.2)

  # Integer breaks for x-axis labels (only those inside the observed range)
  integer_breaks <- seq(floor(x_min_obs), ceiling(x_max_obs), by = 1)
  integer_breaks <- integer_breaks[integer_breaks >= x_min_obs & integer_breaks <= x_max_obs]

  p <- ggplot() +
    # Manual grid lines
    geom_hline(yintercept = y_grid_pos, color = "grey85", linewidth = 0.3) +
    geom_vline(xintercept = x_grid_pos, color = "grey85", linewidth = 0.3) +
    # Posterior credible ribbon
    geom_ribbon(data = plot_df, aes(x = x_stdd, ymin = lo, ymax = hi, fill = "95% Credible interval"), alpha = 0.3) +
```

```
    # Posterior mean line
    geom_line(data = plot_df, aes(x = x_stdd, y = mean, color = "Posterior mean"), linewidth = 1) +
    # Binned observed proportions
    geom_point(data = binned_df, aes(x = x_mid, y = prop, shape = "Observed proportion (binned)"), size = 2, color = "black") +
    # Scales
    scale_y_continuous(breaks = y_grid_pos, limits = c(-0.02, 1.02), name = expression(P(y == 1 ~ "|" ~ D, x))) +
    scale_x_continuous(limits = c(x_min_obs, x_max_obs), breaks = integer_breaks, minor_breaks = x_grid_pos, name = "x (standard
    scale_fill_manual(values = c("95% Credible interval" = "red")) +
    scale_color_manual(values = c("Posterior mean" = "red")) +
    scale_shape_manual(values = c("Observed proportion (binned)" = 16)) +
    guides(
      shape = guide_legend(order = 1),
      color = guide_legend(order = 2),
      fill = guide_legend(order = 3, override.aes = list(alpha = 0.3))
    ) +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      axis.ticks = element_line(color = "black", linewidth = 0.5),
      axis.ticks.length = unit(0.15, "cm"),
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      axis.title = element_text(size = 12),
      axis.text = element_text(size = 10, color = "black"),
      legend.position = "bottom",
      legend.direction = "horizontal",
      legend.box = "horizontal",
      legend.title = element_blank(),
      legend.text = element_text(size = 10, family = "serif"),
      legend.margin = margin(t = 0, b = 0, unit = "pt"),
      legend.spacing.x = unit(5, "pt")
    )

  return(p)
}

plot_posterior(draws_df, x_stdd, y_data)

# function to plot trace plot(s), sampels histogram(s), ACF(s)
plot_various_graphs <- function(my_fit, params_list) {
  draws_array <- as.array(my_fit$draws())
  n_iters <- dim(draws_array)[1]
  n_chains <- dim(draws_array)[2]

  df_list <- list()
  idx <- 1 # index for chains
  for (p in params_list) { # b0, and maybe b1
    for (chain in 1:n_chains) {
      vals <- draws_array[, chain, p]
      df_list[[idx]] <- data.frame(
        Parameter = p,
        Chain = factor(chain),
        Iteration = seq_along(vals),
        Value = as.numeric(vals),
        stringsAsFactors = FALSE
      )
      idx <- idx + 1
    }
  }
  df_long <- do.call(rbind, df_list) # all chains

  # rename parameters as "beta"s
  df_long$Parameter <- factor(df_long$Parameter,
    levels = params_list,
    labels = paste0("beta[", sub("b", "", params_list), "]")
  )

  # trace plots part
  trace_plot <- ggplot(df_long, aes(x = Iteration, y = Value, color = Chain)) +
```

```
    geom_line(alpha = 0.7) +
    facet_wrap(~Parameter, scales = "free_y", ncol = 1, labeller = label_parsed) +
    labs(x = "Iteration", y = "Parameter value") +
    scale_color_manual(values = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73", "4" = "#CC79A7")) +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      strip.background = element_blank(),
      strip.text = element_text(face = "bold", size = 12, family = "serif"),
      legend.position = "bottom",
      legend.title = element_text(size = 10),
      legend.text = element_text(size = 9)
    )

  # occurences histogram part
  hist_plot <- ggplot(df_long, aes(x = Value, fill = Chain, color = Chain)) +
    # relative frequency: each bar = count in bin / total count across all chains
    geom_histogram(aes(y = after_stat(count / sum(count))), alpha = 0.4, bins = 30, position = "identity") +
    # aesthetics only
    facet_wrap(~Parameter, scales = "free", ncol = 1, labeller = label_parsed) +
    labs(x = "Parameter value", y = "Relative frequency") +
    scale_color_manual(values = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73", "4" = "#CC79A7")) +
    # colorblind-friendly palette
    scale_fill_manual(values = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73", "4" = "#CC79A7")) +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      strip.background = element_blank(),
      strip.text = element_text(face = "bold", size = 12, family = "serif"),
      legend.position = "bottom",
      legend.title = element_text(size = 10),
      legend.text = element_text(size = 9)
    )

  # ACFs part
  acf_list <- list()
  for (param in params_list) {
    param_draws <- as.vector(my_fit$draws(param))
    acf_obj <- acf(param_draws, plot = FALSE)
    acf_list[[param]] <- data.frame(
      Lag = as.numeric(acf_obj$lag),
      ACF = as.numeric(acf_obj$acf),
      Parameter = param
    )
  }
  acf_df <- do.call(rbind, acf_list)
  acf_df$Parameter <- factor(acf_df$Parameter,
    levels = params_list,
    labels = paste0("beta[", sub("b", "", params_list), "]")
  )

  acf_plot <- ggplot(acf_df, aes(x = Lag, y = ACF)) +
    geom_hline(yintercept = 0, color = "black", linewidth = 0.4) +
    geom_segment(aes(xend = Lag, yend = 0), color = "black", linewidth = 0.7) +
    facet_wrap(~Parameter, ncol = 1, labeller = label_parsed) +
    labs(x = "Lag", y = "ACF") +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      strip.background = element_blank(),
      strip.text = element_text(face = "bold", size = 12, family = "serif"),
      legend.position = "none",
      legend.text = element_text(size = 9, family = "serif")
    )
  return(list(acf = acf_plot, trace = trace_plot, density = hist_plot))
}
graphs <- plot_various_graphs(fit, c("b0", "b1"))
```

```r
print(graphs$trace)
print(graphs$density)
print(graphs$acf)

# POINT 4
# uncertainty on pi_new is the red interval before, just for a fixed x
x_new_original <- 0 # fixed value
x_new_std <- (x_new_original - x_mean) / x_std # standardize it
# posterior draws of pi_new
pi_new_draws <- plogis(draws_df$b0 + draws_df$b1 * x_new_std) # pi(x=0  )
# ie for all chain samples, and fixed x_new, plot posterior's pi
# posterior mean and 95% credible interval
pi_mean <- mean(pi_new_draws)
pi_ci <- quantile(pi_new_draws, c(0.025, 0.975))
cat(pi_mean, pi_ci)
# 0.4156503 0.3174578 0.5160177
central_point <- (pi_ci[1] + pi_ci[2]) / 2
cat(central_point)
# 0.4195013
# approximately equal to mean; so, can use central interval:
delta_half <- (-pi_ci[1] + pi_ci[2]) / 2
cat(delta_half)
# 0.09792069

# note: posterior predictive probability P(y_new=1 | x_new, D) = pi_mean
# histogram of pi_new draws
par(family = "serif")
hist(pi_new_draws,
  breaks = 30, probability = TRUE,
  xlab = expression(pi[new]), col = "grey50",
  main = "", border = "white", yaxt = "n", xaxt = "n"
)
axis(1, at = seq(0, 1, by = 0.2), labels = seq(0, 1, by = 0.2))
axis(2, las = 1)
lines(density(pi_new_draws), col = "red", lwd = 2)
legend("topright",
  legend = c("Histogram", "KDE estimate"),
  fill = c("grey50", NA), border = c("black", NA),
  col = c(NA, "red"), lty = c(NA, 1), lwd = c(NA, 2),
  bty = "n", cex = 0.9
)

# INTERCEPT ONLY MODEL
res <- build_fit_stan_model("C:/Users/Libero/uni/1B/adv_stats/assignment_3/intercept_only_model.stan", stan_data_sdd)
i_mod <- res$mod
i_fit <- res$fit
i_fit$summary(c("b0"))
# # A tibble: 1 × 10
#   variable   mean median    sd   mad     q5   q95  rhat ess_bulk ess_tail
#   <chr>     <dbl>  <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>    <dbl>    <dbl>
# 1 b0       -0.202 -0.200 0.183 0.179 -0.505 0.100  1.00    3011.    3961.
i_fit$cmdstan_diagnose()
# (...)
# (everything ok)
i_draws_df <- as_draws_df(i_fit$draws()) # draws from posterior
plot_posterior(i_draws_df, x_stdd, y_data)
i_graphs <- plot_various_graphs(i_fit, c("b0"))
print(i_graphs$trace)
print(i_graphs$density)
print(i_graphs$acf)

# MODEL COMPARISON
# extract log-likelihood matrix (draws x N)
log_lik <- fit$draws("log_lik", format = "matrix")
# compute LOO object for both models
loo <- loo(log_lik)
i_log_lik <- i_fit$draws("log_lik", format = "matrix")
i_loo <- loo(i_log_lik)
comp <- loo_compare(loo, i_loo) # compare the two models based on ELPD statistics
print(loo)
```

```r
print(i_loo)
# (...)
# diagnostics ok: in particular k < 0.7
print(comp) # difference in ELPD and standard error
#       elpd_diff se_diff
# model1  0.0       0.0
# model2 -9.9       4.2

# note: if elpd_diff > 2*se_diff, difference is considered significant; here 9.9 > 2* 4.2
```

Declaration of AI use: AI assistance was used solely for improving the appearance of plots and for improving readability (no code generation or analysis).